

5. Longest Palindromic Substring Medium

Acceptance: 37.1% | [LeetCode](#)

Given a string `s`, return *the longest palindromic substring* in `s`.

Example 1:

```
**Input:** s = "babad"  
**Output:** "bab"  
**Explanation:** "aba" is also a valid answer.
```

Example 2:

```
**Input:** s = "cbdd"  
**Output:** "bb"
```

Constraints:

- `1 <= s.length <= 1000`
- `s` consist of only digits and English letters.

Claude Opus Solution

Longest Palindromic Substring

1. Intuition

The **key insight** is recognizing the recursive structure of palindromes:

A string is a palindrome if its **outer characters match** AND its **inner substring is also a palindrome**.

For example, "abcba" is a palindrome because `s[0] == s[4]` (`'a' == 'a'`) AND the inner part "bcb" is a palindrome.

This naturally leads to two approaches: - **Dynamic Programming**: Build up palindrome knowledge from smaller substrings (like the reference solution). - **Expand Around Center**: Treat each position as a potential palindrome center and expand outward.

The **Expand Around Center** approach is more elegant and space-efficient, so I'll use it as the primary solution.

The crucial observation: every palindrome has a **center**. For a string of length `n`, there are `2n - 1` possible centers: - `n` centers for **odd-length** palindromes (centered on a character, e.g., "aba"). - `n - 1` centers for **even-length** palindromes (centered between two characters, e.g., "abba").

2. Approach (Pseudocode)

```
function longestPalindrome(s):  
    if s is empty: return ""  
  
    start, end = 0, 0 # track best palindrome boundaries  
  
    for each center i from 0 to n-1:  
        # Case 1: odd-length palindrome (single character center)  
        len1 = expandFromCenter(s, i, i)  
        # Case 2: even-length palindrome (gap between i and i+1)  
        len2 = expandFromCenter(s, i, i+1)  
  
        maxLen = max(len1, len2)  
        if maxLen > (end - start + 1):  
            update start and end based on center i and maxLen  
  
    return s[start : end+1]  
  
function expandFromCenter(s, left, right):  
    # Expand while characters match and we're in bounds  
    while left >= 0 and right < n and s[left] == s[right]:  
        left -= 1  
        right += 1  
    return right - left - 1 # length of palindrome found
```

3. Python Solution

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        if not s:
            return ""

        # start, end mark the boundaries of the best palindrome found so far
        start, end = 0, 0

        # Helper: expand outward from a center as long as it stays a palindrome.
        # Returns the LENGTH of the longest palindrome with this center.
        def expand_from_center(left: int, right: int) -> int:
            # Keep expanding while in bounds and characters match
            while left >= 0 and right < len(s) and s[left] == s[right]:
                left -= 1
                right += 1
            # When the loop exits, left and right have gone one step too far.
            # The valid palindrome spans (left+1) to (right-1) inclusive,
            # so its length is (right - 1) - (left + 1) + 1 = right - left - 1
            return right - left - 1

        # Try every possible center
        for i in range(len(s)):
            # Case 1: odd-length palindrome -> single character center at i
            len1 = expand_from_center(i, i)

            # Case 2: even-length palindrome -> center is the gap between i and i+1
            len2 = expand_from_center(i, i + 1)

            # Take the larger of the two palindromes found at this center
            max_len = max(len1, len2)

            # Update the answer if we found a longer palindrome
            if max_len > end - start + 1:
                # Recompute boundaries from the center i and the palindrome length.
                # For length L centered at i:
                #   left = i - (L - 1) // 2
                #   right = i + L // 2
                # This formula works for both odd and even lengths.
                start = i - (max_len - 1) // 2
                end = i + max_len // 2

        # Return the substring (end is inclusive, so +1)
        return s[start:end + 1]
```

Walkthrough of boundary formula

For center i and length L : - **Odd** (e.g., $L=3$, center on char i): $\text{left} = i - 1$, $\text{right} = i + 1$ ✓ - **Even** (e.g., $L=4$, center in gap after i): $\text{left} = i - 1$, $\text{right} = i + 2$ ✓

The formula $\text{start} = i - (L-1)//2$, $\text{end} = i + L//2$ handles both cases uniformly.

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(n^2)$	We try $2n - 1$ centers, and each expansion can take up to $O(n)$ time in the worst case (e.g., "aaaa...").
Space	$O(1)$	Only a constant number of pointers/variables (excluding the output string).

Comparison with the reference DP solution: - The DP solution also runs in $O(n^2)$ time but uses $O(n^2)$ space for the memo table. - Expand-around-center achieves the **same time with only $O(1)$ space** — a strict improvement.

5. Alternative Approaches

a) Dynamic Programming (Reference Solution)

- Build a 2D table $\text{dp}[i][j] = \text{"is } s[i..j] \text{ a palindrome?"}$.
- Transition: $\text{dp}[i][j] = (s[i] == s[j]) \text{ and } \text{dp}[i+1][j-1]$.

- **Time:** $O(n^2)$, **Space:** $O(n^2)$. Easy to reason about but uses more memory.

b) Manacher's Algorithm (Optimal)

- A clever linear-time algorithm that uses previously computed palindrome lengths (via symmetry around a "center") to avoid redundant expansions.
- Typically transforms the string by inserting separators (e.g., #) to handle odd/even cases uniformly.
- **Time:** $O(n)$, **Space:** $O(n)$. Optimal but complex to implement — overkill for $n \leq 1000$.

c) Brute Force

- Check every substring for being a palindrome.
- **Time:** $O(n^3)$ ($O(n^2)$ substrings $\times$ $O(n)$ palindrome check). Too slow; avoid.

Recommendation: For this problem's constraints ($n \leq 1000$), the **Expand Around Center** approach is ideal — it's simple, fast enough, and uses minimal space. Reserve Manacher's for cases needing strict $O(n)$ performance.

Codex Solution

1. Intuition

A palindrome reads the same forward and backward.

The key insight is:

Every palindrome has a **center**.

There are two possible types of centers:

1. Odd-length palindrome

Example: "racecar"

Center is one character: 'e'

2. Even-length palindrome

Example: "abba"

Center is between two characters: 'b' and 'b'

So for every index in the string, we can try to expand outward and find the longest palindrome centered there.

For example, in "babad" :

```
b a b a d
  ^
center at 'a'
expand outward:
a
bab
```

"bab" is a palindrome.

This avoids checking every substring manually.

2. Approach

We use the **expand around center** technique.

Pseudocode

```
function longestPalindrome(s):
    best_start = 0
    best_length = 1

    for each index i in s:
        # Case 1: odd-length palindrome centered at i
        left = i
        right = i
        expand while left/right are valid and s[left] == s[right]
        update best palindrome if needed

        # Case 2: even-length palindrome centered between i and i + 1
        left = i
```

```
right = i + 1
expand while left/right are valid and s[left] == s[right]
update best palindrome if needed

return substring from best_start with best_length
```

3. Python Solution

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        # Length of the input string
        n = len(s)

        # Since constraints say n >= 1, we can safely initialize
        # the best palindrome as the first character.
        best_start = 0
        best_len = 1

        def expand_from_center(left: int, right: int) -> None:
            """
            Expands outward from the given center while the characters match.

            For odd-length palindromes:
                left == right

            For even-length palindromes:
                right == left + 1
            """
            nonlocal best_start, best_len

            # Keep expanding as long as:
            # 1. left and right are inside the string
            # 2. characters at both ends are equal
            while left >= 0 and right < n and s[left] == s[right]:
                current_len = right - left + 1

                # If this palindrome is longer than the best one seen so far,
                # update the answer.
                if current_len > best_len:
                    best_start = left
                    best_len = current_len

                # Expand outward
                left -= 1
                right += 1

        # Try every index as a possible palindrome center.
        for i in range(n):
            # Odd-length palindrome centered at i
            # Example: "racecar", center is 'e'
            expand_from_center(i, i)

            # Even-length palindrome centered between i and i + 1
            # Example: "abba", center is between the two 'b's
            expand_from_center(i, i + 1)

        # Return the longest palindromic substring found
        return s[best_start:best_start + best_len]
```

4. Complexity Analysis

Let $n = \text{len}(s)$.

Time Complexity: $O(n^2)$

For each index, we expand around two centers:

1. Odd-length center
2. Even-length center

Each expansion can take up to $O(n)$ time in the worst case.

Example worst case:

```
s = "aaaaaaaaa"
```

Every center expands almost across the whole string.

So the total time complexity is:

```
 $O(n \text{ centers} * n \text{ expansion}) = O(n^2)$ 
```

Space Complexity: $O(1)$

We only store:

- best_start
- best_len
- left
- right

No extra array or table is used.

Therefore:

```
 $O(1)$ 
```

5. Alternative Approaches

1. Dynamic Programming

We can define:

```
dp[i][j] = True if s[i:j+1] is a palindrome
```

Transition:

```
s[i] == s[j] and dp[i + 1][j - 1] == True
```

This also runs in:

```
Time:  $O(n^2)$   
Space:  $O(n^2)$ 
```

This is similar to the reference solution.

2. Manacher's Algorithm

Manacher's Algorithm solves the problem in linear time:

```
Time:  $O(n)$   
Space:  $O(n)$ 
```

However, it is more complex and usually unnecessary for LeetCode constraints where:

```
n <= 1000
```

The expand-around-center solution is simpler and efficient enough.